

Role: Full Stack Developer (React + Node.js + SQL)

Purpose: 1-Hour Technical Interview (Q&A + Code Examples)

Prepared for: Technical Interviewers

Section 1: Introduction & Warm-Up (3 Questions)

Q1: Tell me about yourself and your experience as a full stack developer.

A1: Assess communication, ability to summarize skills, clarity, and structure.

Q2: What technologies have you worked with recently?

A2: Expect React, Node.js, SQL/NoSQL, Git, REST APIs, etc.

Q3: Describe a challenging project and how you solved key technical issues.

A3: Tests problem-solving, debugging skills, and practical implementation.

Section 2: HTML & CSS (15 Questions)

Q1: Difference between block-level and inline elements.

A1: Block-level elements take full width and start on a new line (e.g., <div>). Inline elements take only required width and do not break lines (e.g.,).

Q2: CSS box model and its components.

A2: Content → Padding → Border → Margin.

Q3: Create a 3-column layout using Flexbox.

A3:

```
<div style="display:flex;">
  <div>Column 1</div>
  <div>Column 2</div>
  <div>Column 3</div>
</div>
```

Q4: Center a div vertically and horizontally using CSS.

Q4:

```
<div style="display:flex; justify-content:center; align-items:center; height:100vh;">  
  <div>Centered</div>  
</div>
```

Q5: Difference between relative, absolute, and fixed positioning.

A5: Relative: normal flow; Absolute: relative to nearest ancestor; Fixed: relative to viewport.

Q6: Responsive design techniques.

A6: Use media queries, flexible units (%/em/rem), CSS Flexbox/Grid.

Q7: Flexbox vs CSS Grid.

A7: Flexbox: 1D layouts; Grid: 2D layouts.

Q8: Pseudo-classes in CSS.

A8: Used to define special states of elements, e.g., :hover, :focus.

Q9: CSS specificity rules.

A9: ID > Class > Element, inline > everything.

Q10: Difference between inline, inline-block, and block.

A10: Inline-block allows width/height, inline does not.

Q11: What are semantic HTML tags?

A11: Tags that give meaning, e.g., <header>, <article>, <footer>.

Q12: Difference between relative units (em, rem) and px.

A12: em/rem scale relative, px absolute.

Q13: Optimize CSS performance.

A13: Minify, combine files, avoid deep nesting.

Q14: Z-index usage.

A14: Controls stacking order of elements.

Q15: CSS transitions and animations.

A15: Smoothly change properties over time using transition or @keyframes.

Section 3: JavaScript & React (15 Questions)

Q1: Difference between == and ===.

A1: == checks equality with type coercion; === checks equality strictly.

Q2: React component types.

A2: Class Components and Functional Components (Hooks preferred).

Q3: Virtual DOM explanation.

A3: In-memory DOM for efficient updates to the real DOM.

Q4: useEffect hook usage.

A4:

```
useEffect(() => {  
  // side effect code  
}, [dependencies]);
```

Q5: Props vs state.

A5: Props: parent passed, read-only. State: internal, mutable.

Q6: Closure example.

A6:

```
function outer() {  
  let x = 10;  
  return function inner() { return x; };  
}
```

Q7: Remove duplicates from array.

A7:

```
const arr = [1,2,2,3];  
const unique = [...new Set(arr)];
```

Q8: Event delegation.

A8: Attach listener to parent, handle events for child elements.

Q9: Async/await example.

```
A9: async function fetchData() {  
  const res = await fetch(url);  
}
```

Q10: Difference between var, let, const.

A10: var: function-scoped; let/const: block-scoped; const immutable reference.

Q11: React Context API usage.

```
A11:  
const MyContext = React.createContext();
```

Q12: Explain React key prop.

A12: Helps identify elements in lists for efficient re-rendering.

Q13: React hooks explanation.

A13: Functions to manage state/lifecycle in functional components.

Q14: Stateful vs stateless components.

A14: Stateful has internal state, stateless does not.

Q15: Debounce function example.

```
A15:  
function debounce(fn, delay){  
  let timer;  
  return (...args) => {  
    clearTimeout(timer);  
    timer = setTimeout(()=>fn(...args), delay);  
  };  
}
```

Section 4: Node.js & Backend (15 Questions)

Q1: What is Node.js?

A1: JavaScript runtime outside browser, event-driven, non-blocking I/O.

Q2: Explain event loop.

A2: Handles asynchronous tasks efficiently.

Q3: module.exports vs exports.

A3: module.exports is returned; exports is reference object.

Q4: Streams in Node.js.

A4: Readable, Writable, Duplex, Transform.

Q5: Callbacks vs Promises vs async/await.

A5: Callbacks: traditional; Promises: chainable; async/await: cleaner syntax.

Q6: Create a basic Express.js route.

A6: `app.get('/api', (req, res) => res.send('Hello'));`

Q7: Middleware in Express.js.

A7:

```
function(req,res,next){  
  console.log(req.url);  
  next();  
}
```

Q8: Error handling in Node.js.

A8: try/catch, error middleware, event listeners.

Q9: REST API principles.

A9: Stateless, uniform interface, client-server separation.

Q10: Authentication vs Authorization.

A10: Authentication: who you are; Authorization: what you can access.

Q11: Difference between synchronous and asynchronous.

A11: Sync: blocks execution; Async: non-blocking.

Q12: npm usage.

A12: Node package manager to manage dependencies.

Q13: Clustering in Node.js.

A13: Run multiple processes to utilize CPU cores.

Q14: Debug Node.js app.

A14: Use console.log, Node inspector, breakpoints.

Q15: What is CORS?

A15: Cross-Origin Resource Sharing policy to allow/disallow requests.

Section 5: SQL & Databases (15 Questions)

Q1: What is SQL?

A1: Structured Query Language used to manage relational databases.

Q2: DDL vs DML vs DQL.

A2: DDL: define structure, DML: data changes, DQL: queries.

Q3: DELETE vs TRUNCATE vs DROP.

A3: DELETE: remove rows, TRUNCATE: remove all rows, DROP: remove table.

Q4: Normalization 1NF,2NF,3NF.

A4: 1NF: atomic, 2NF: full dependency, 3NF: no transitive dependency.

Q5: WHERE vs HAVING.

A5: WHERE filters rows before aggregation, HAVING filters after aggregation.

Q6: INNER JOIN vs LEFT JOIN vs RIGHT JOIN.

A6: INNER: matching rows; LEFT: all left; RIGHT: all right.

Q7: Primary key vs foreign key.

A7: Primary: unique; Foreign: references PK.

Q8: Index usage.

A8: Speeds up query searches.

Q9: Transactions.

A9: Ensure atomic operations with commit/rollback.

Q10: Stored procedures.

A10: Reusable SQL code blocks.

Q11: GROUP BY usage.

A11: Aggregate rows by column(s).

Q12: ORDER BY usage.

A12: Sort query results.

Q13: Aggregate functions.

A13: SUM, COUNT, AVG, MIN, MAX.

Q14: Subqueries.

A14: Query within a query.

Q15: CHAR vs VARCHAR.

A15: CHAR: fixed length; VARCHAR: variable length.